

Sex Classification using Periocular Images

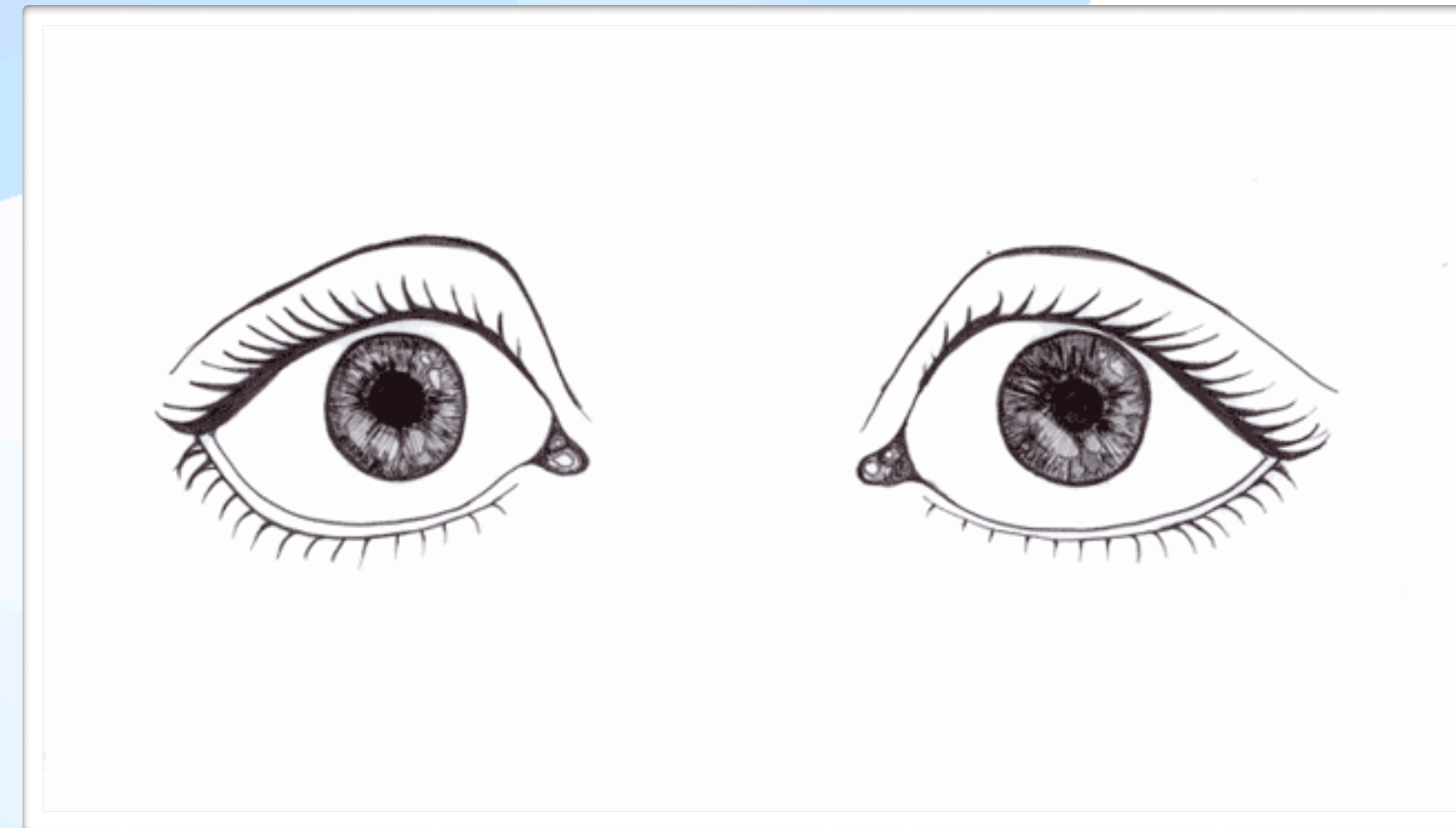
& Deep Convolutional Neural Networks

Cody Hill | October 10, 2024

[https://github.com/chill0121/Periocular Sex Classification](https://github.com/chill0121/Periocular_Sex_Classification)

Project Summary

- Eye image dataset to build Deep Convolutional Neural Network (DCNN) classifiers for sex identification.
- **Key Focuses:**
 - Multiple DCNN architectures.
 - Training image augmentation.
 - Comparison of model performance using image augmentation.
 - Model-extracted features most relevant for sex classification.



Motivations

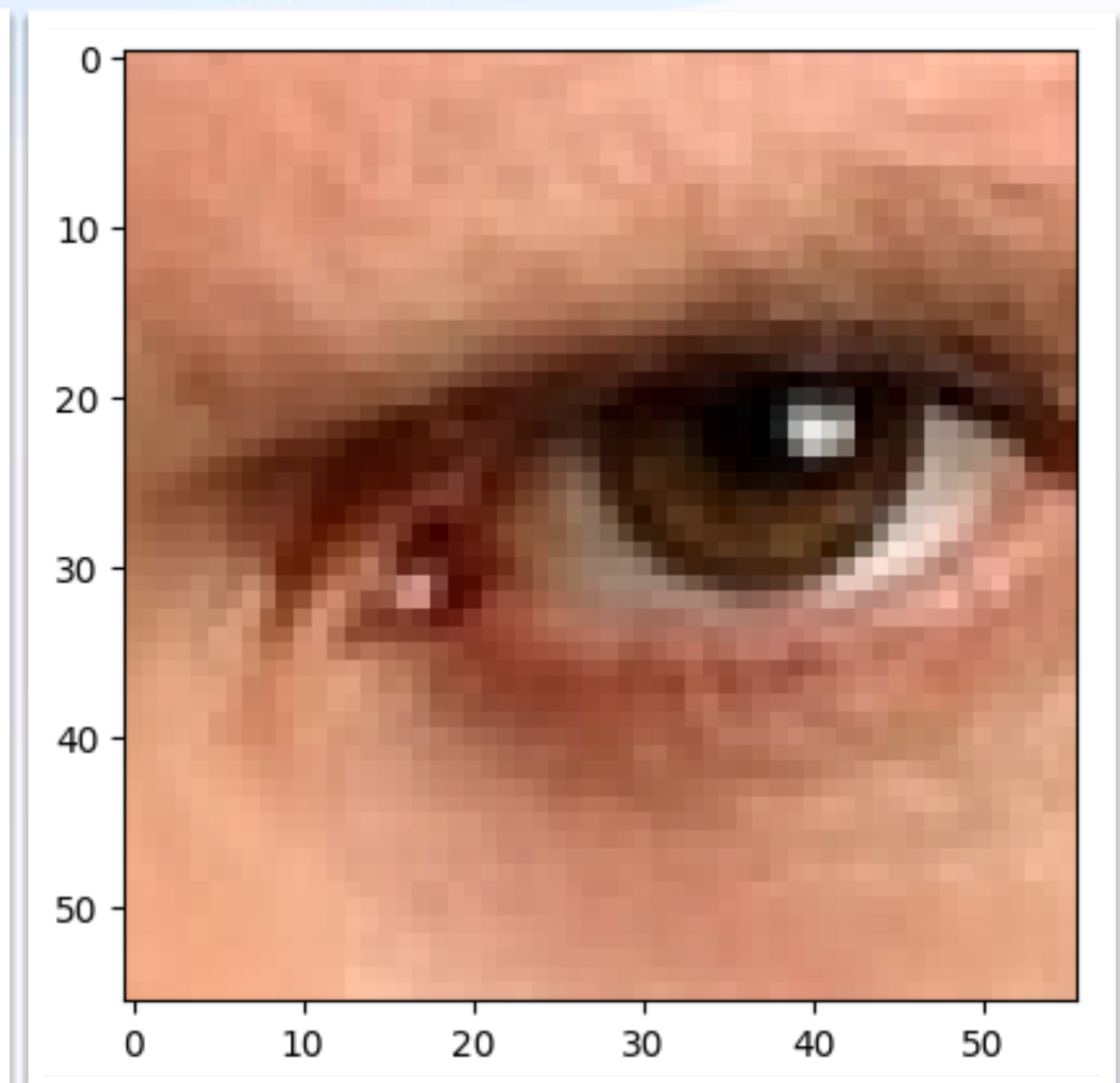
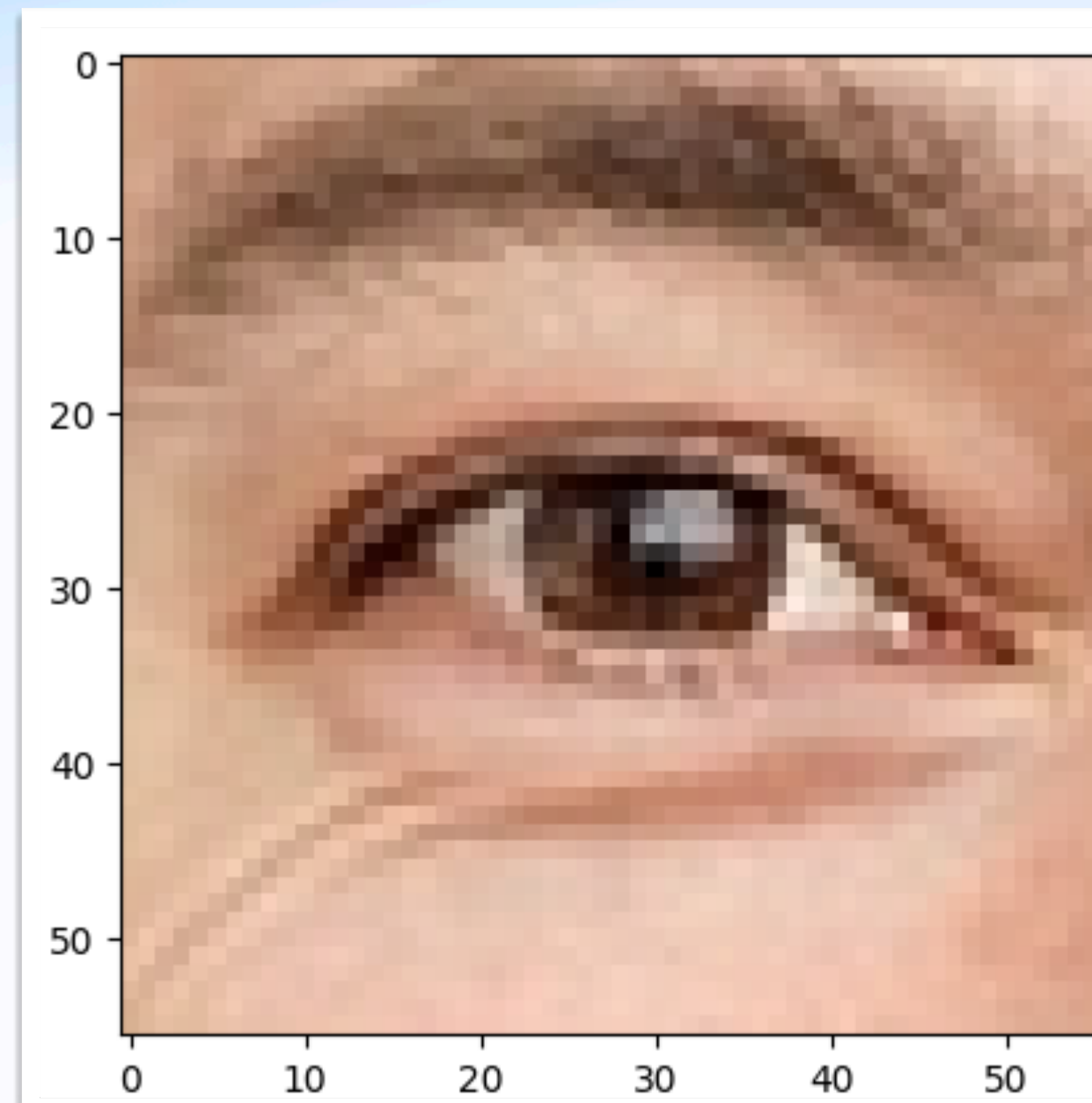
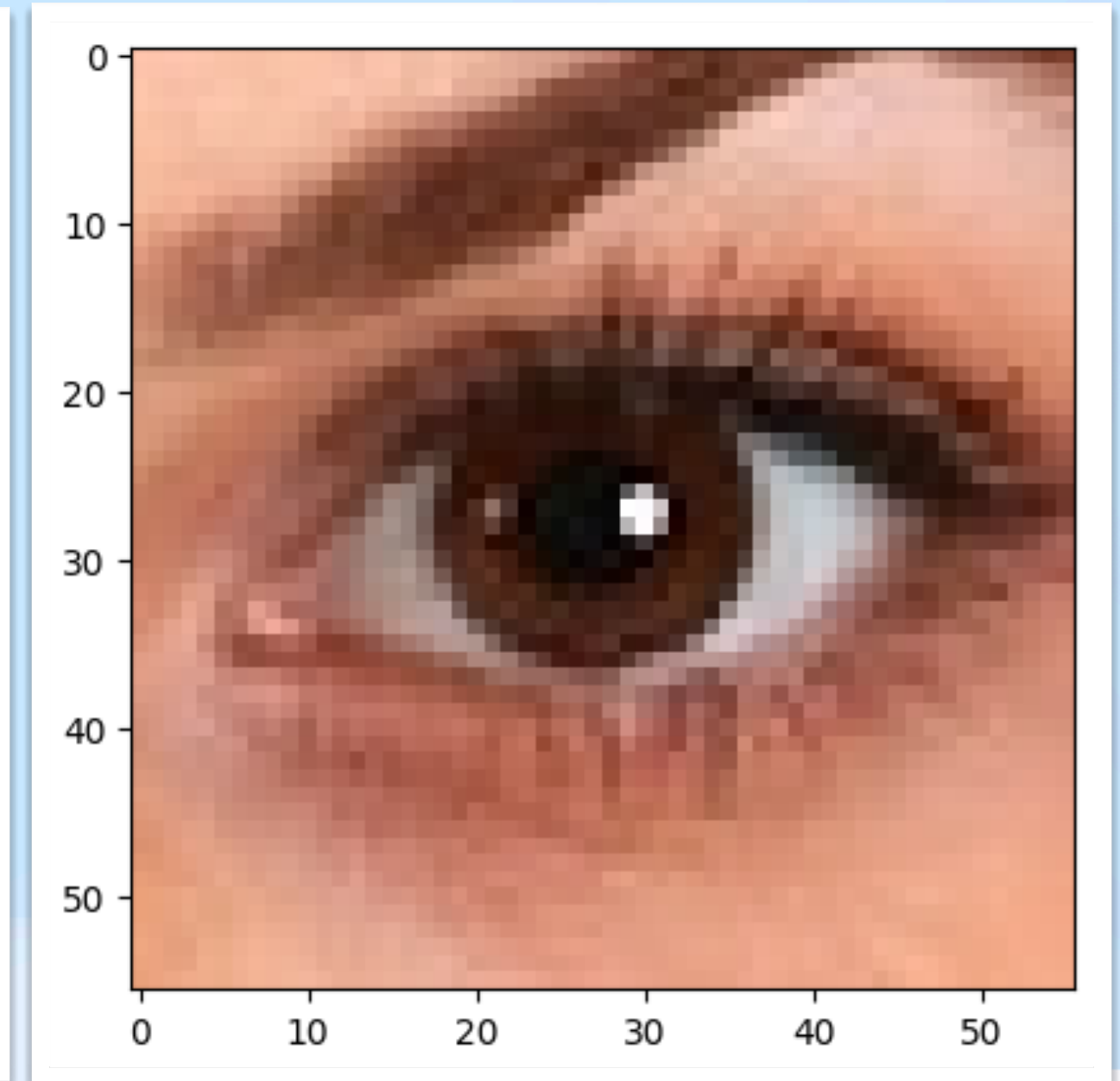
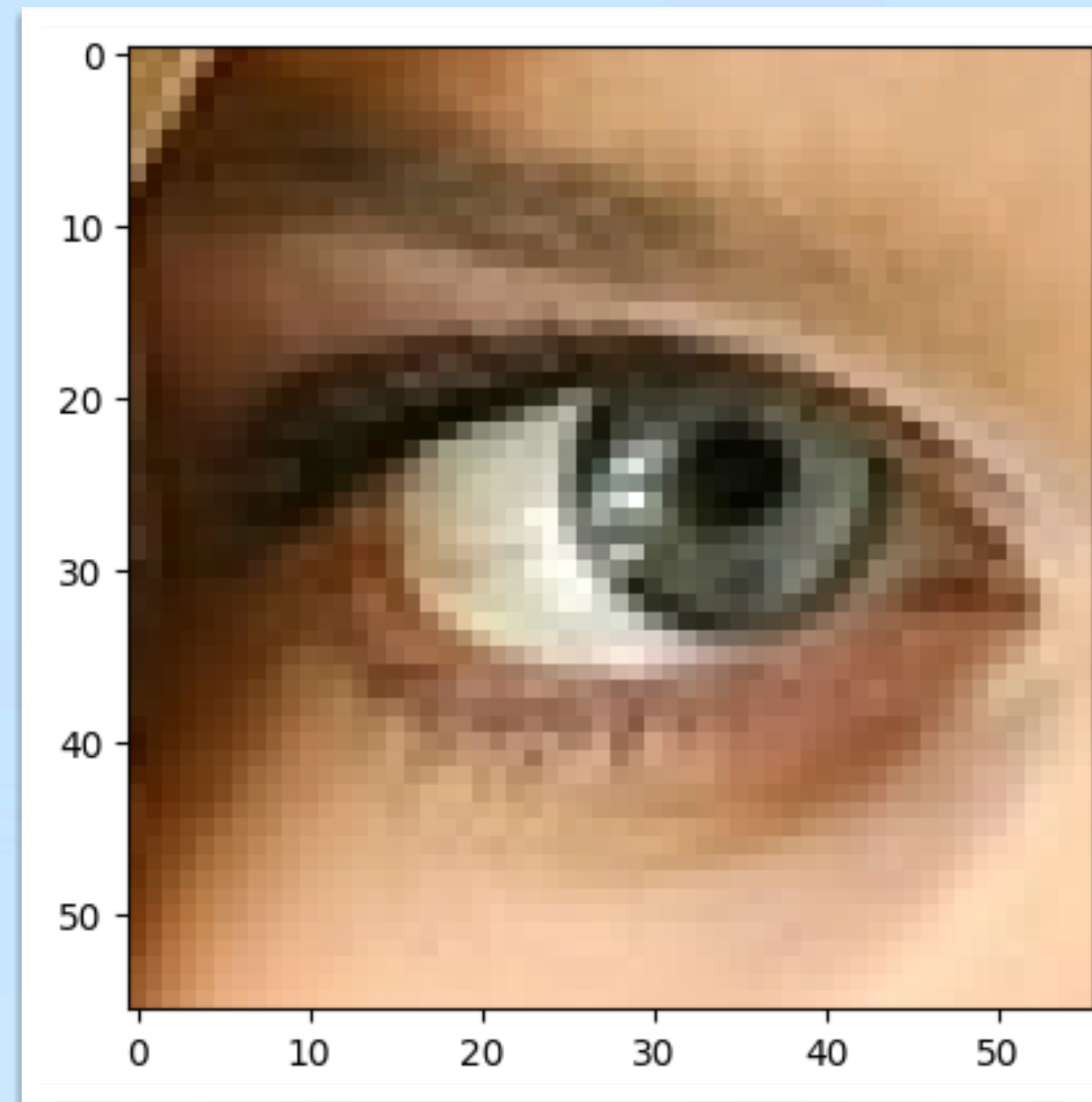
- Applications in security
 - Biometric authentication with partial or low-resolution data.
- Enhance identity verification systems.
- Addresses the need for adaptable, privacy-conscious, and efficient systems for real-world scenarios with limited or non-ideal data.

Dataset

Source Information

<https://www.kaggle.com/datasets/pavelbiz/eyes-rtte/data>

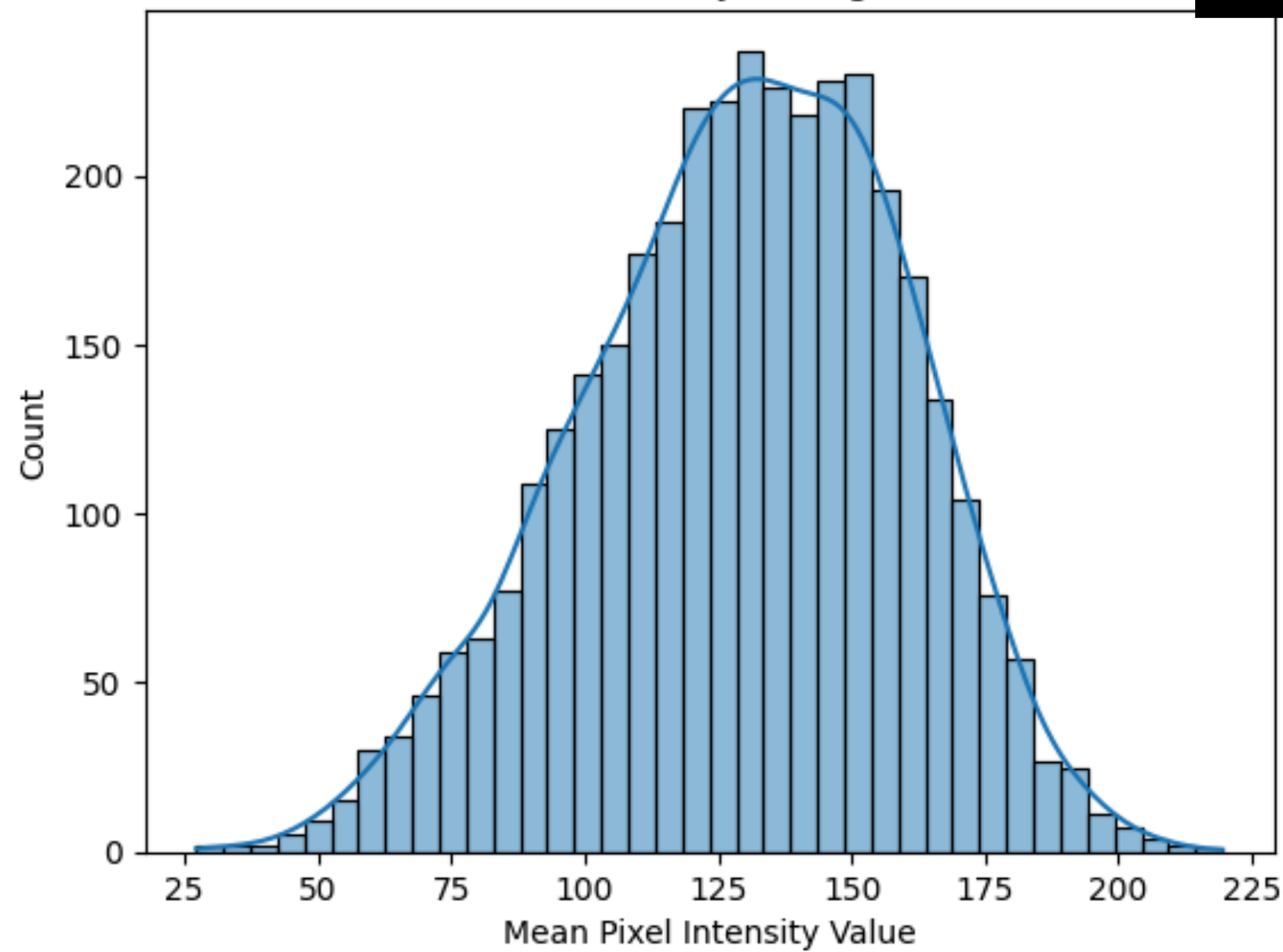
- Web scraped using OpenCV Haar Eye Cascade
- IMDB-type Website
- Dataset Information:
 - 5,203 Female
 - 6,324 Male
 - Various Image Sizes
 - Balanced during processing



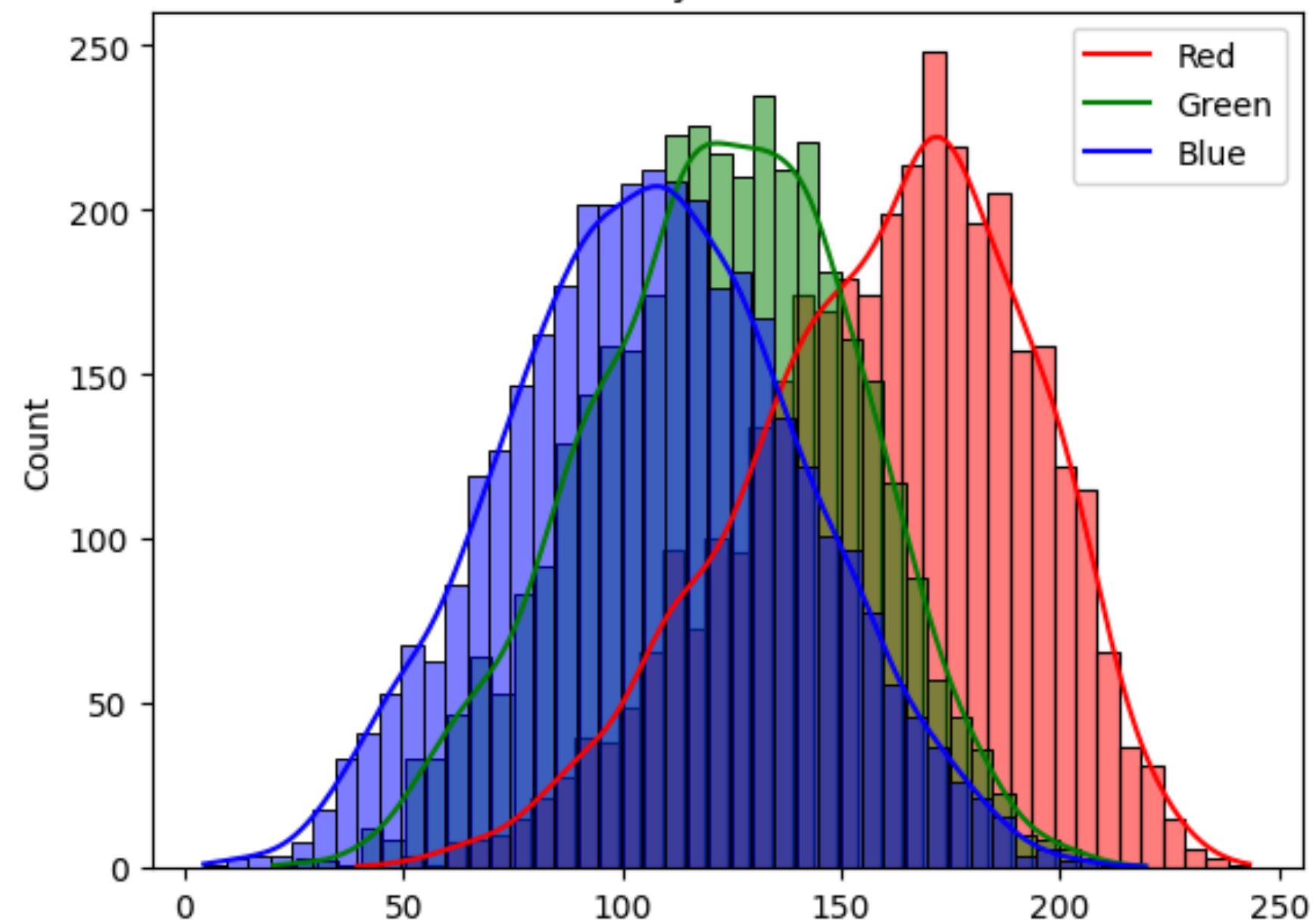
Exploratory Data Analysis (EDA)

Analyzing Pixel Intensity Values

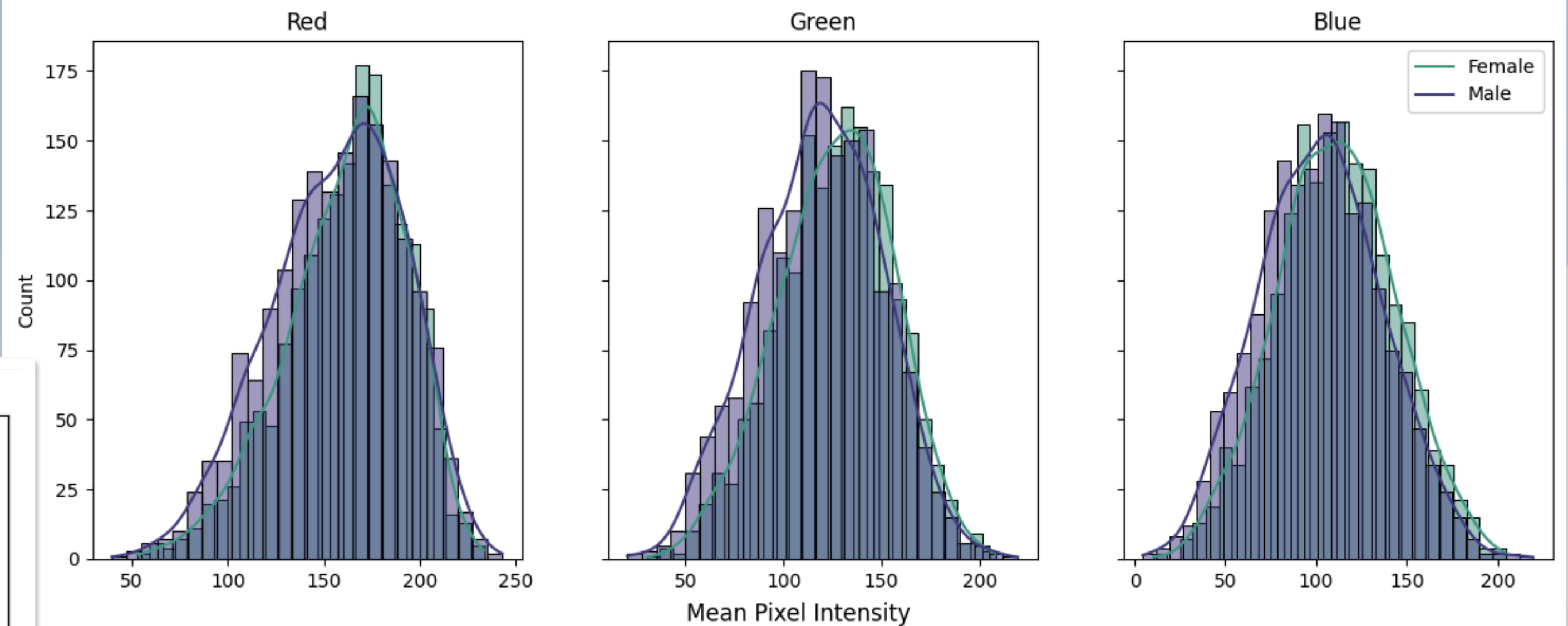
Pixel Intensity Histogram



Mean Pixel Intensity of Each Channel (RGB)

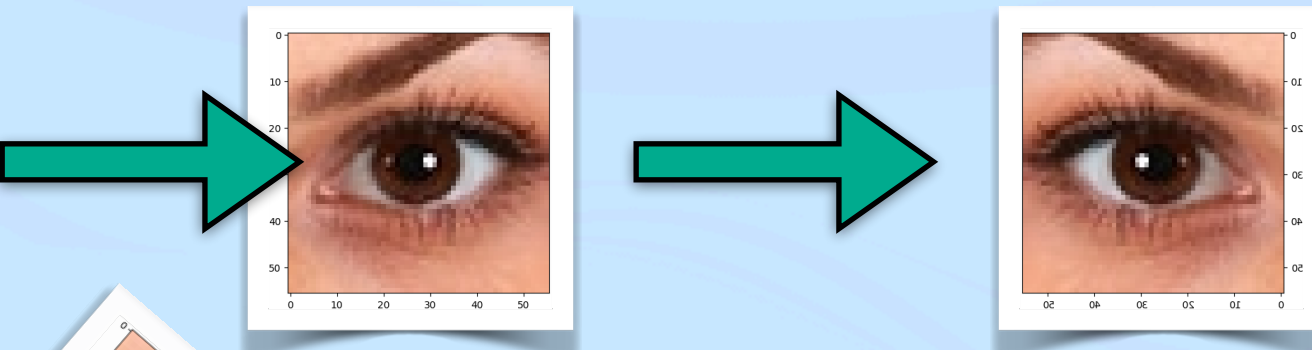


Color Channel Distributions by Class Label

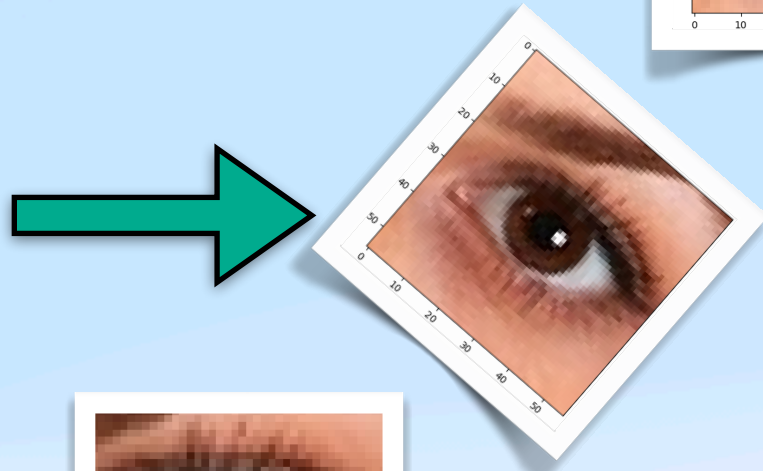


Training Image Augmentation

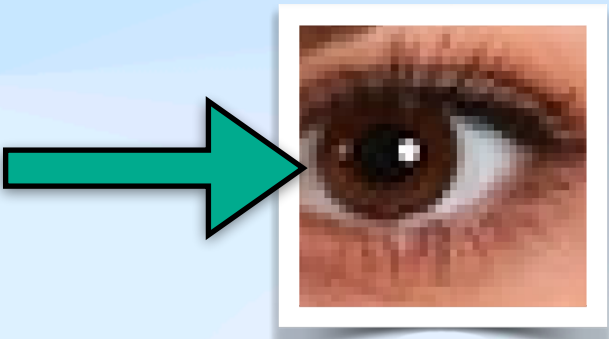
- Random Flip (Horizontal)



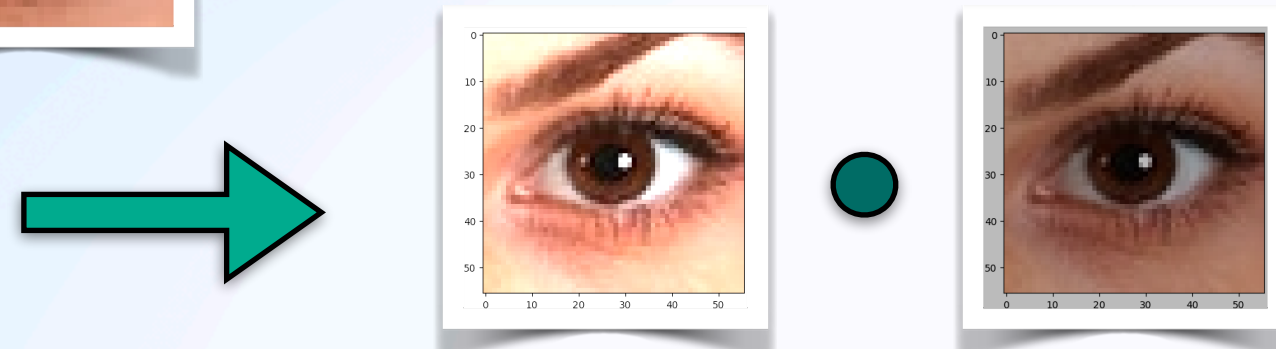
- Random Rotation



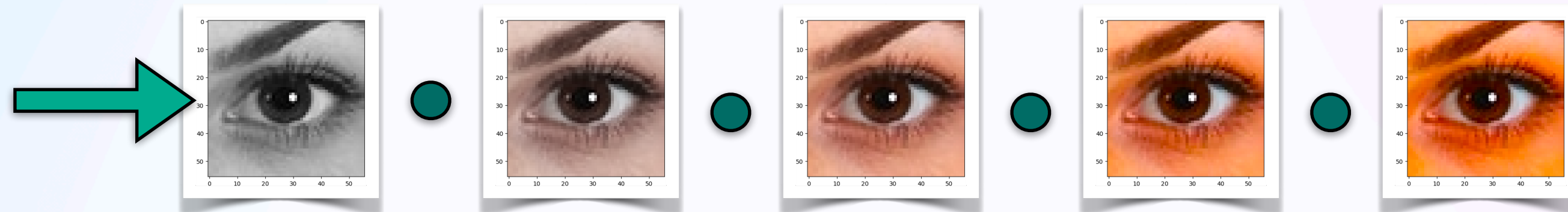
- Random Crop



- Random Brightness



- Random Saturation

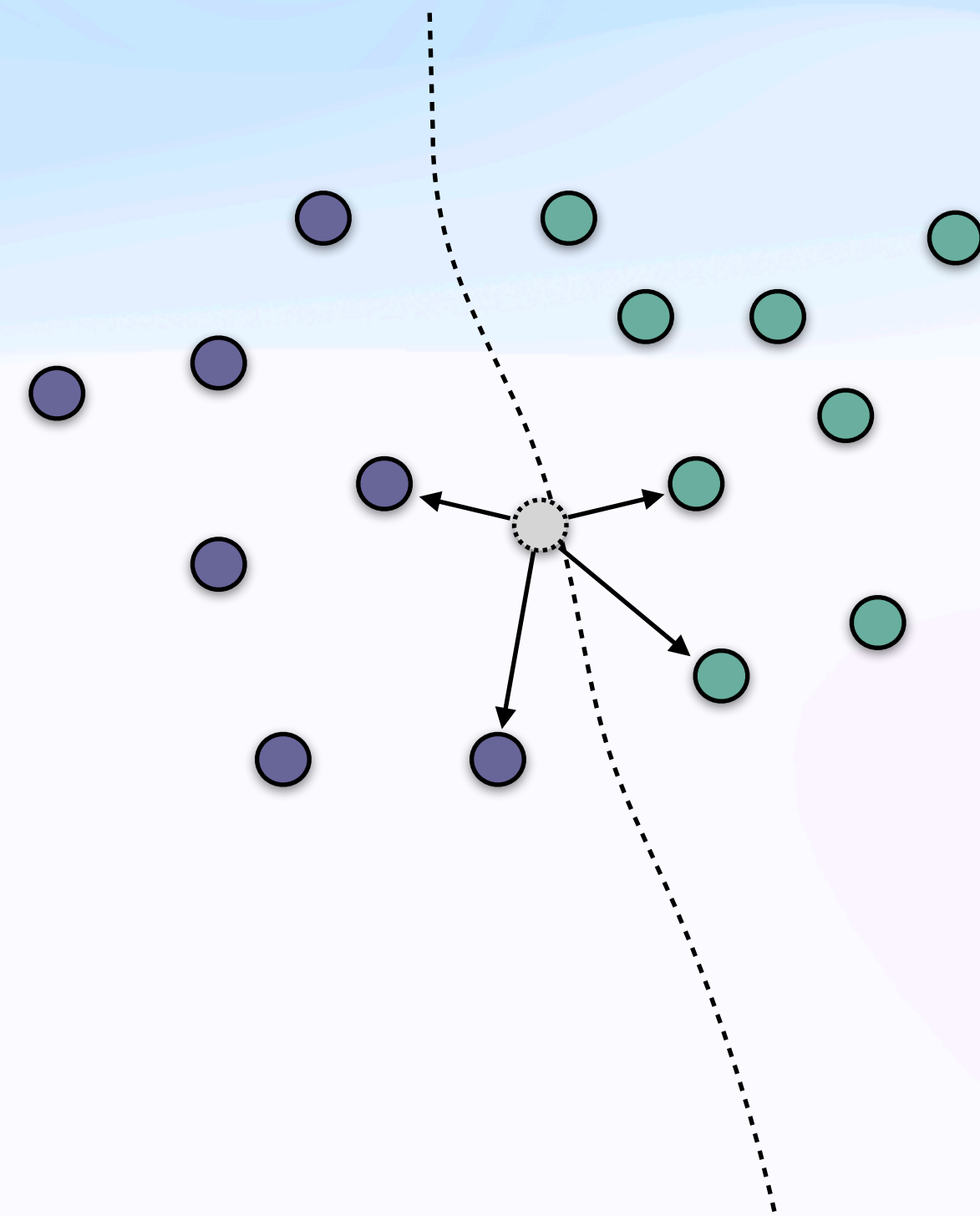


Baseline Models

- Random Chance

$$\frac{1}{n_{classes}} = \frac{1}{2}$$

- K-Nearest Neighbors (KNN)



Baseline Models

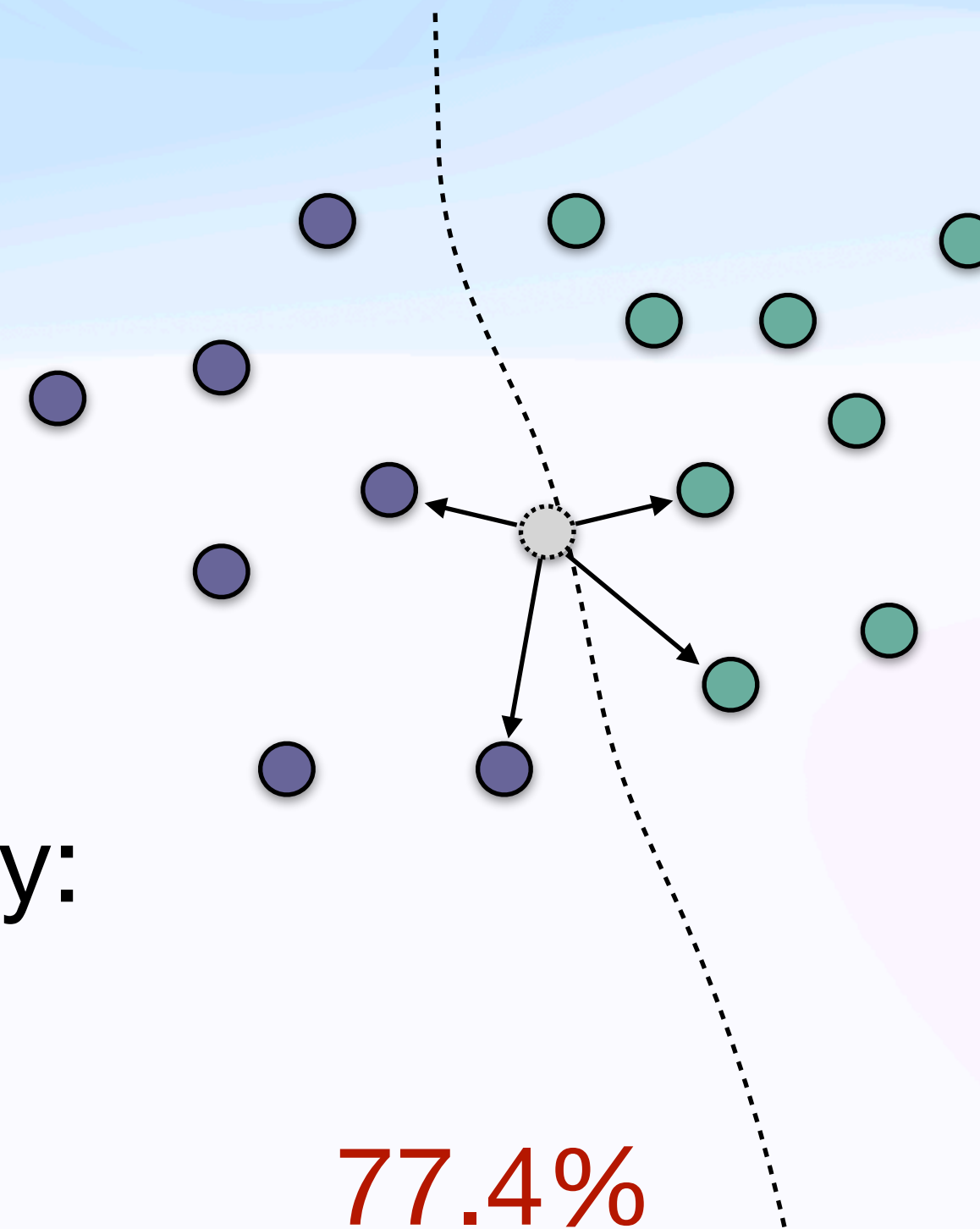
- Random Chance

$$\frac{1}{n_{classes}} = \frac{1}{2}$$

Validation Set Accuracy:

50%

- K-Nearest Neighbors (KNN)



77.4%

“Deep” Learning Models

Shallow Feedforward Neural Network (FNN)

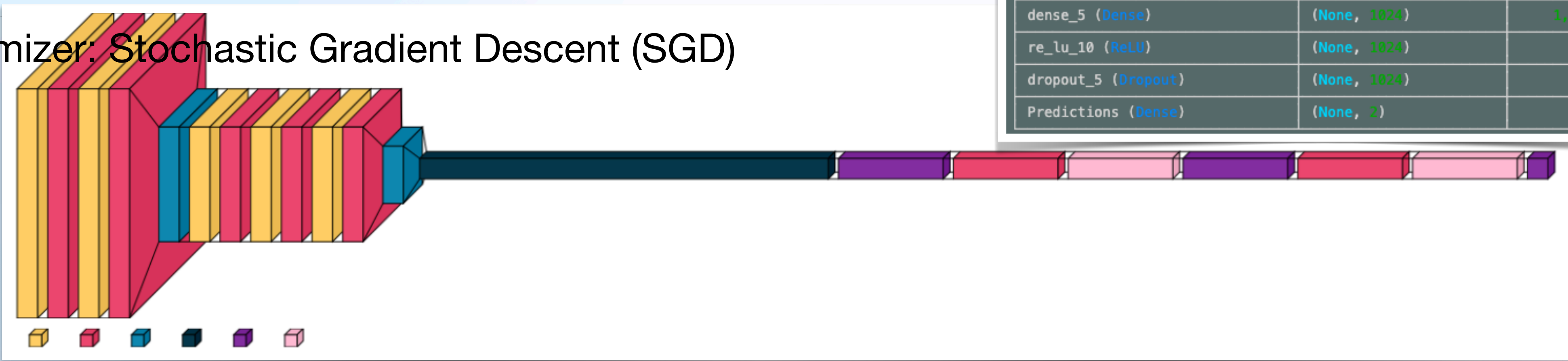
- Shallow Dense Network
- ReLU Activation
- 4,883,330 Parameters
- Optimizer: Adam
- Loss: Cross-Entropy

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 9408)	0
dense (Dense)	(None, 512)	4,817,408
re_lu (ReLU)	(None, 512)	0
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 128)	65,664
re_lu_1 (ReLU)	(None, 128)	0
dropout_1 (Dropout)	(None, 128)	0
Predictions (Dense)	(None, 2)	258

Deep Learning Models

Deep-ish Convolutional Neural Network (CNN)

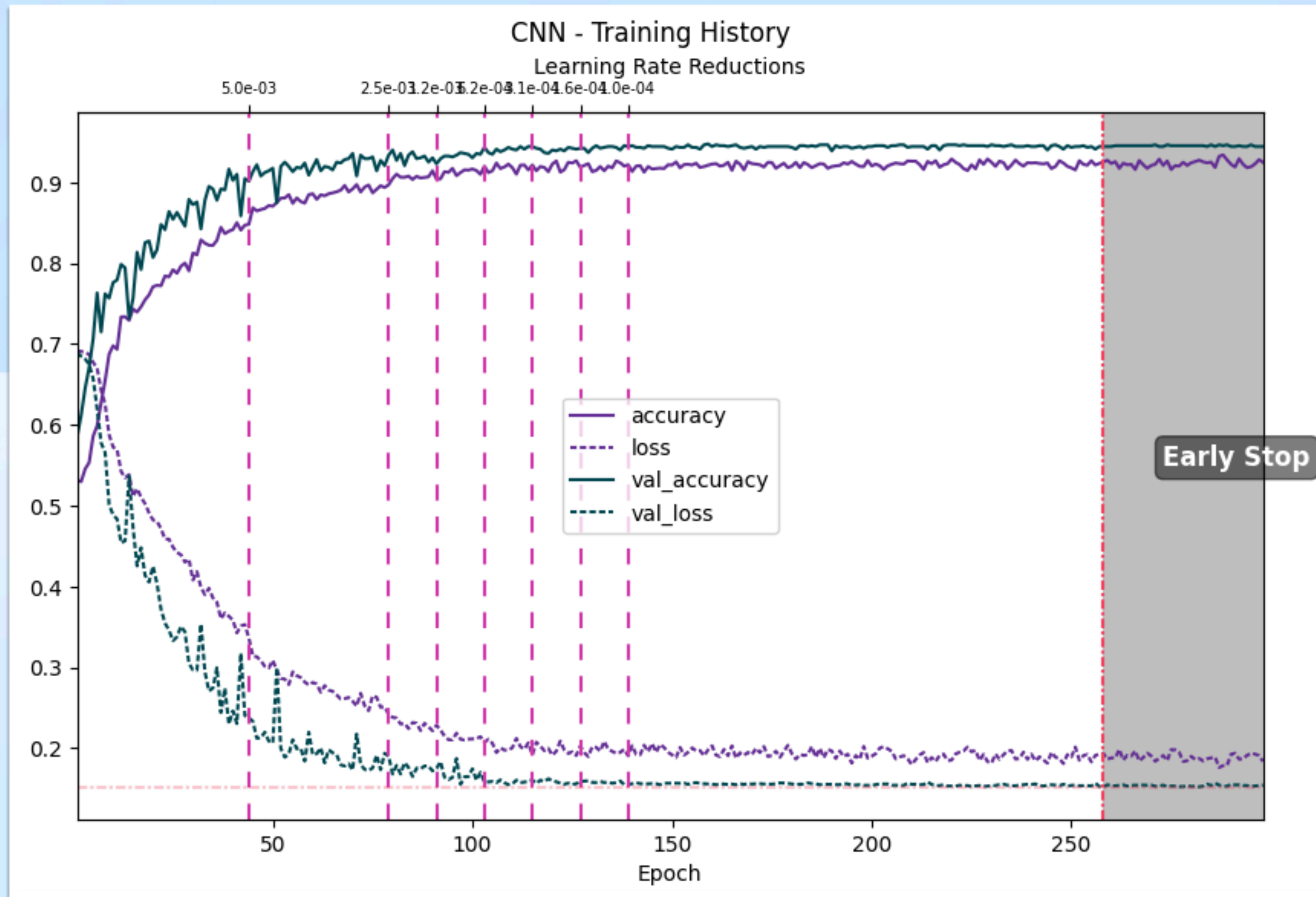
- Convolutional Feature Extractor
- 3x3 Stacked Kernels
- Max Pooling (Stride 2 Downsampling)
- Dense Network Classifier
- ReLU Activation
- 7,501,106 Parameters
- Optimizer: Stochastic Gradient Descent (SGD)



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 56, 56, 16)	448
re_lu_4 (ReLU)	(None, 56, 56, 16)	0
conv2d_1 (Conv2D)	(None, 56, 56, 16)	2,320
re_lu_5 (ReLU)	(None, 56, 56, 16)	0
max_pooling2d (MaxPooling2D)	(None, 28, 28, 16)	0
conv2d_2 (Conv2D)	(None, 28, 28, 32)	4,640
re_lu_6 (ReLU)	(None, 28, 28, 32)	0
conv2d_3 (Conv2D)	(None, 28, 28, 32)	9,248
re_lu_7 (ReLU)	(None, 28, 28, 32)	0
conv2d_4 (Conv2D)	(None, 28, 28, 32)	9,248
re_lu_8 (ReLU)	(None, 28, 28, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 32)	0
flatten_2 (Flatten)	(None, 6272)	0
dense_4 (Dense)	(None, 1024)	6,423,552
re_lu_9 (ReLU)	(None, 1024)	0
dropout_4 (Dropout)	(None, 1024)	0
dense_5 (Dense)	(None, 1024)	1,049,600
re_lu_10 (ReLU)	(None, 1024)	0
dropout_5 (Dropout)	(None, 1024)	0
Predictions (Dense)	(None, 2)	2,050

Deep Learning Models

Deep-ish Convolutional Neural Network (CNN)



- Early Stopping
 - Patience: 40
- Reduce Learning Rate:
 - Patience 5
 - Cooldown 8
 - Min LR: 0.0001

Deep Learning Models

Deep Convolutional Neural Network (DCNN)

- Inspired by ResNet-50
- 3x3 Stacked Kernels
- Batch Normalization
- ReLU Activation
- Global Pooling to Dense Predictor
- 21,306,626 Parameters
- Optimizer: Stochastic Gradient Descent (SGD)

```
# Build model.
def build_dcnn(input_shape = (img_size, img_size, 3)):
    inputs = tf.keras.Input(shape = input_shape)

    # Initial convolutional layer. Reduced kernel size here from default 7,7 because the input images are small.
    x = tf.keras.layers.Conv2D(64, kernel_size = (5, 5), strides = (2, 2), padding = 'same')(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)
    x = tf.keras.layers.MaxPooling2D(pool_size = (3, 3), strides = (2, 2), padding = 'same')(x)

    # First residual block (3 blocks).
    for _ in range(3):
        x = residual_block(x, 64)

    # Second residual block (4 blocks).
    x = residual_block(x, 128, downsample = True)
    for _ in range(3):
        x = residual_block(x, 128)

    # Third residual block (6 blocks).
    x = residual_block(x, 256, downsample = True)
    for _ in range(5):
        x = residual_block(x, 256)

    # Fourth residual block (3 blocks).
    x = residual_block(x, 512, downsample = True)
    for _ in range(2):
        x = residual_block(x, 512)

    # Global average pooling layer and final output prediction.
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    outputs = tf.keras.layers.Dense(2, name = 'Predictions')(x)

    # Create model.
    model = tf.keras.Model(inputs, outputs)
    return model
```

```
def residual_block(x, filters, downsample = False):
    shortcut = x
    strides = (2, 2) if downsample else (1, 1)

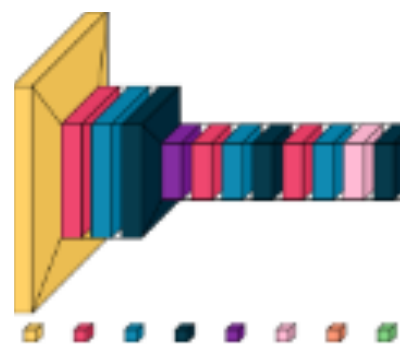
    # First convolution layer.
    x = tf.keras.layers.Conv2D(filters, kernel_size = (3, 3), strides = strides, padding = 'same')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    # Second convolution layer.
    x = tf.keras.layers.Conv2D(filters, kernel_size = (3, 3), padding = 'same')(x)
    x = tf.keras.layers.BatchNormalization()(x)

    # Add convolution if downsample == True.
    if downsample:
        shortcut = tf.keras.layers.Conv2D(filters, kernel_size = (1, 1), strides = (2, 2), padding = 'same')(shortcut)
        shortcut = tf.keras.layers.BatchNormalization()(shortcut)

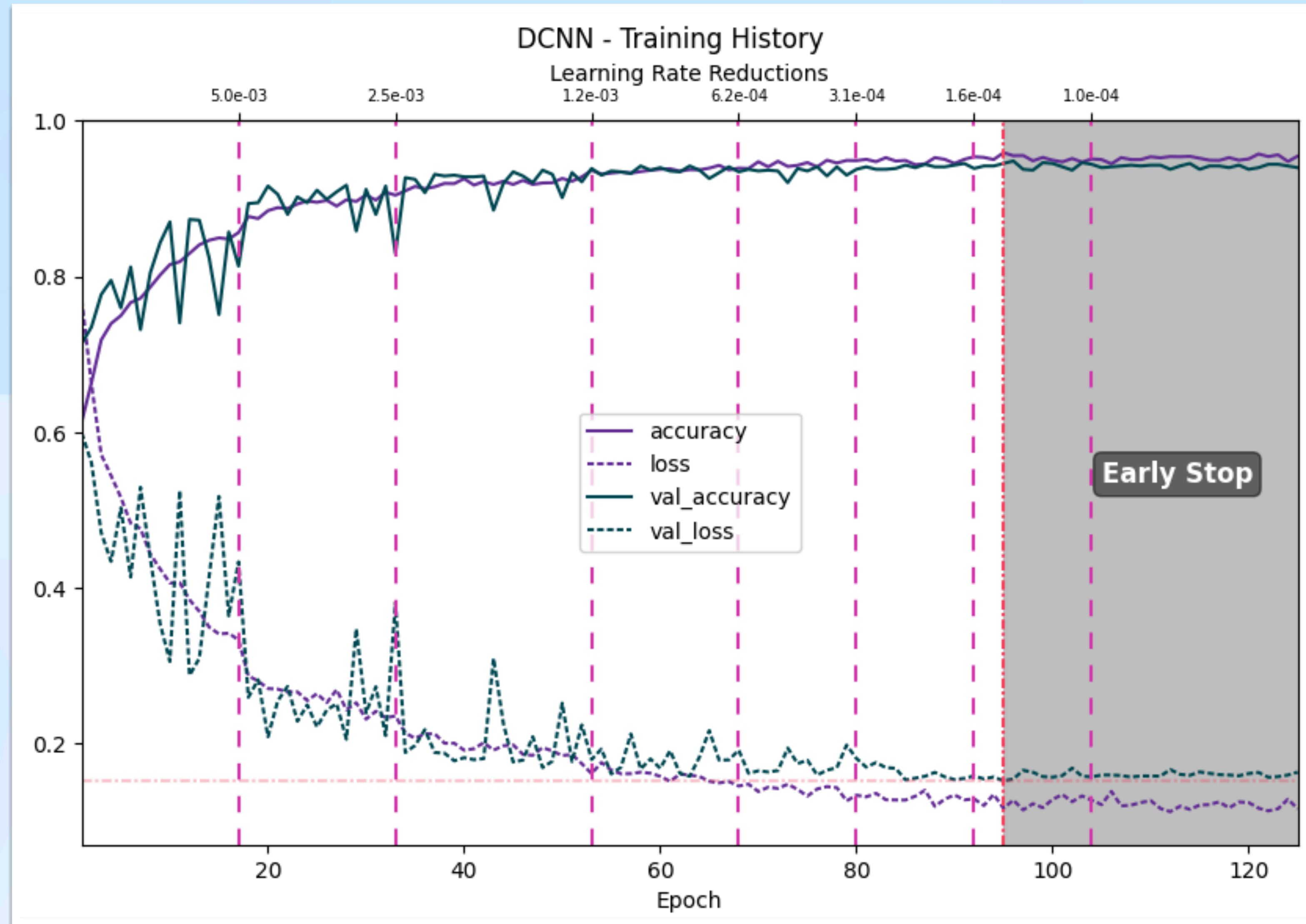
    # Add the shortcut (input) to the output.
    x = tf.keras.layers.Add()([x, shortcut])
    x = tf.keras.layers.ReLU()(x)

    return x
```



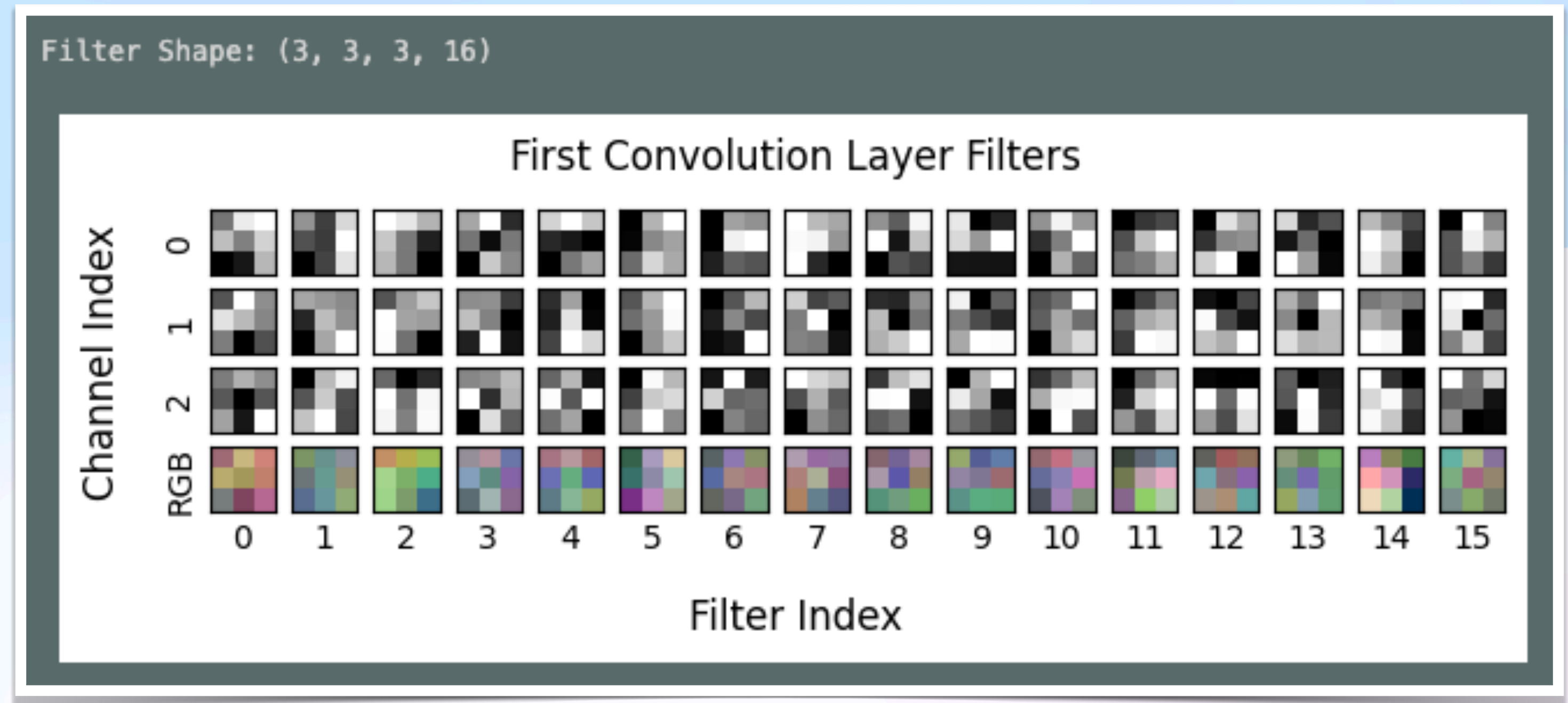
Deep Learning Models

Deep Convolutional Neural Network (DCNN)

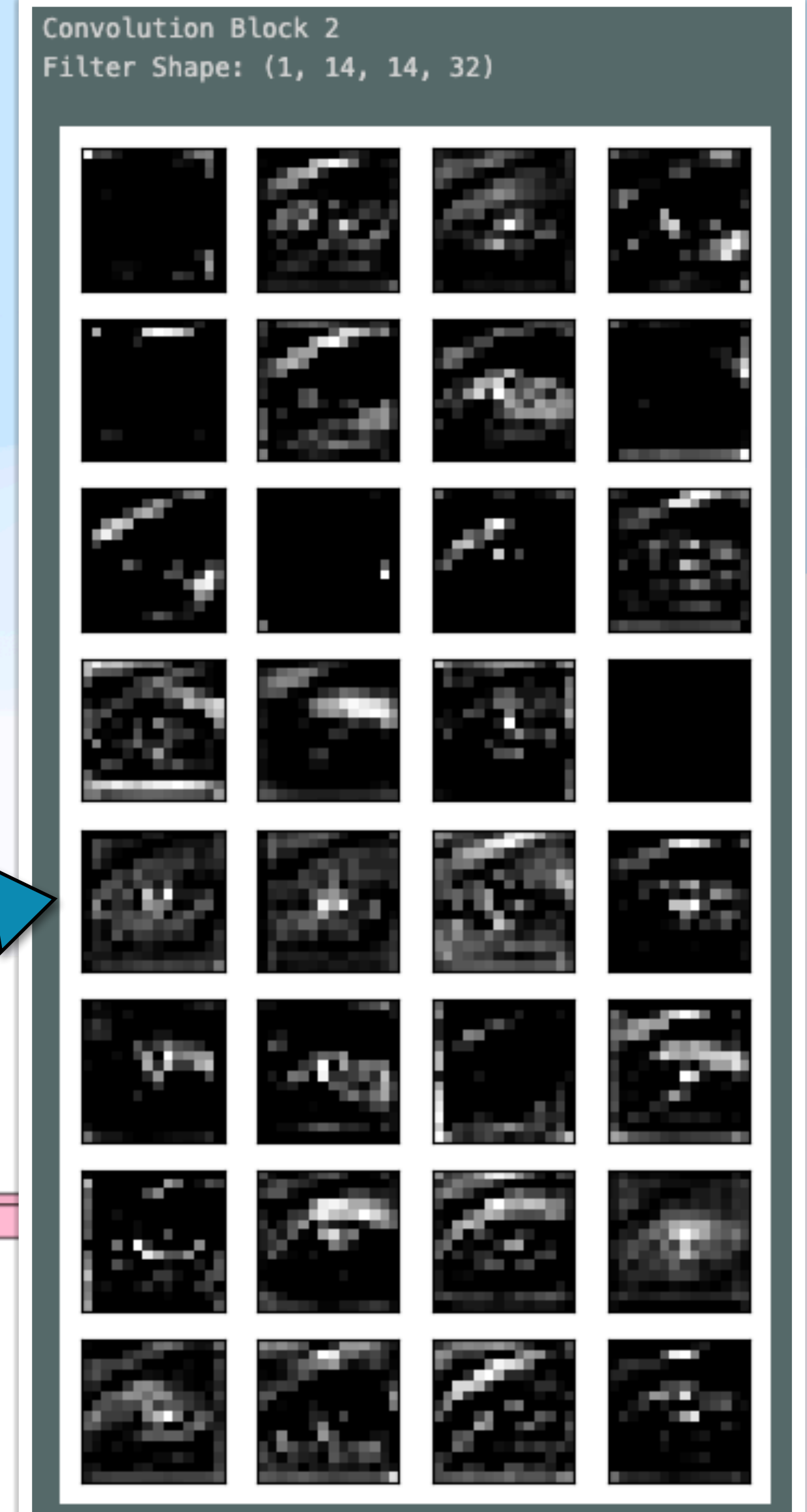
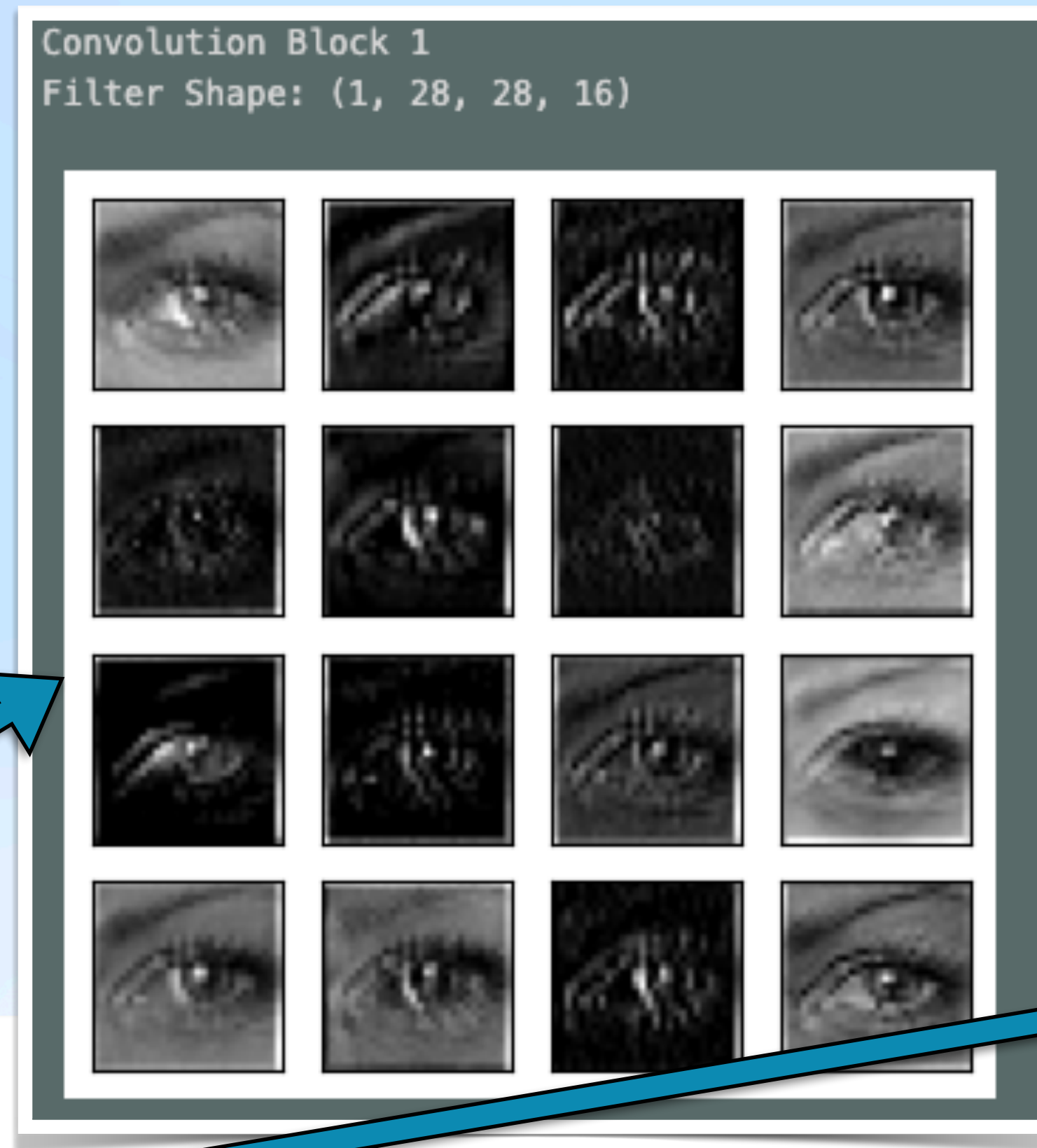
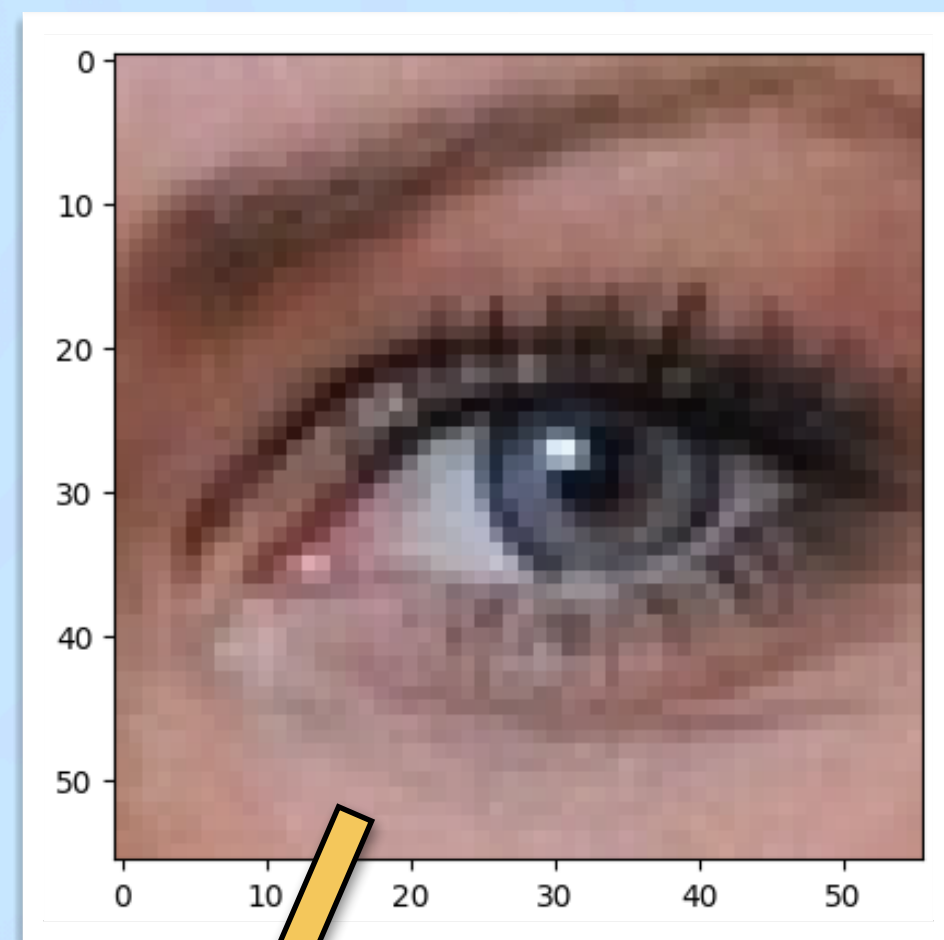


Model Analysis | Feature Extraction

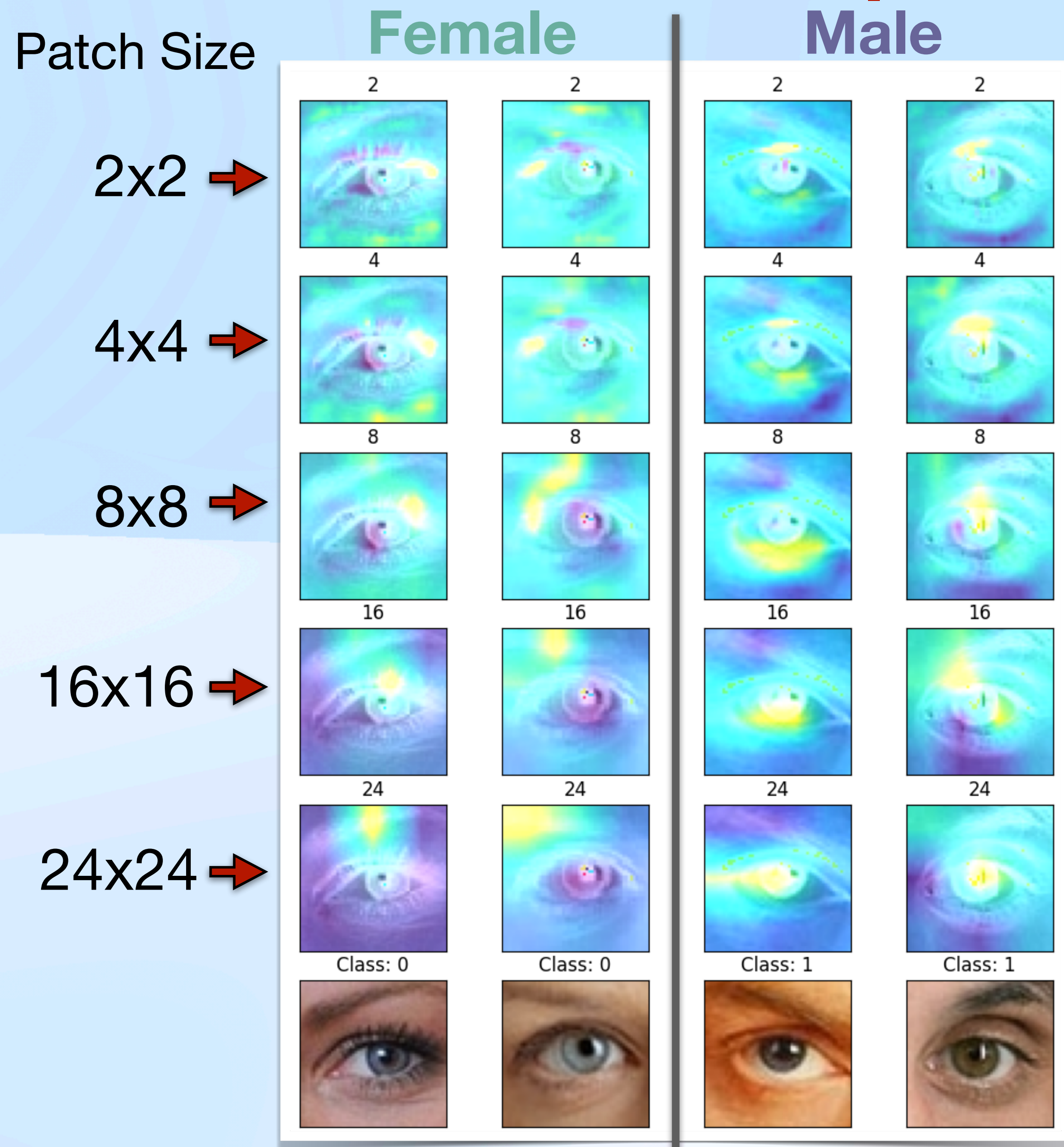
- By extracting weights from convolution layers we can visualize filters.
- Visualizes 16, 3x3 kernels.



Model Analysis | Feature Extraction



Model Analysis | Occlusion Sensitivity



- Iterates across image, occluding sections with patches.
- Maps changes in probabilities of class with occlusion.
- Parameters:
 - Patch Size

Model Analysis | Principal Component Analysis (PCA)

Model: **CNN**

Layer 13 - First layer after feature extraction.

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Conv2D(3,3)

Conv2D(3,3)

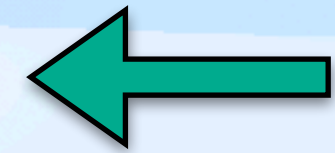
MaxPooling(2,2)

Flatten()

Dense(1024)

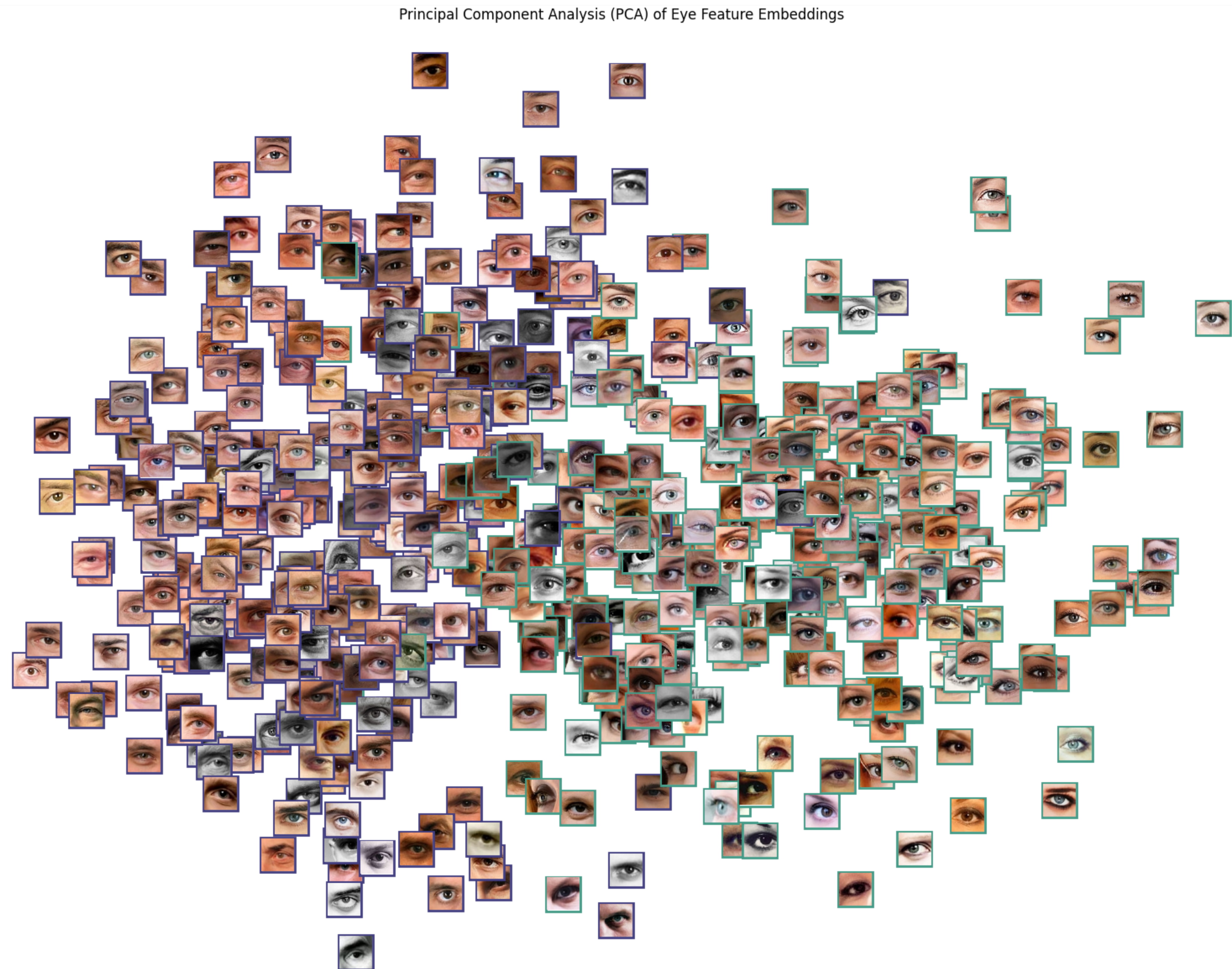
Dense(1024)

Dense(2)



Explained Variance Ratios:

- PC1: 20.4%
- PC2: 9.1%



Model Analysis | Principal Component Analysis (PCA)

Model: **CNN**

Layer 13 - First layer
after feature extraction.

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

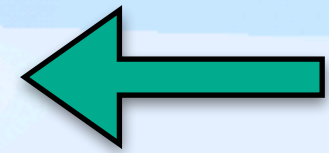
Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Flatten()

Dense(1024)



Dense(1024)

Dense(2)

Explained Variance Ratios:

- PC1: 20.4%
- PC2: 9.1%



Model Analysis | Principal Component Analysis (PCA)

Model: **CNN**

Layer 16 - Last layer before prediction.

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Flatten()

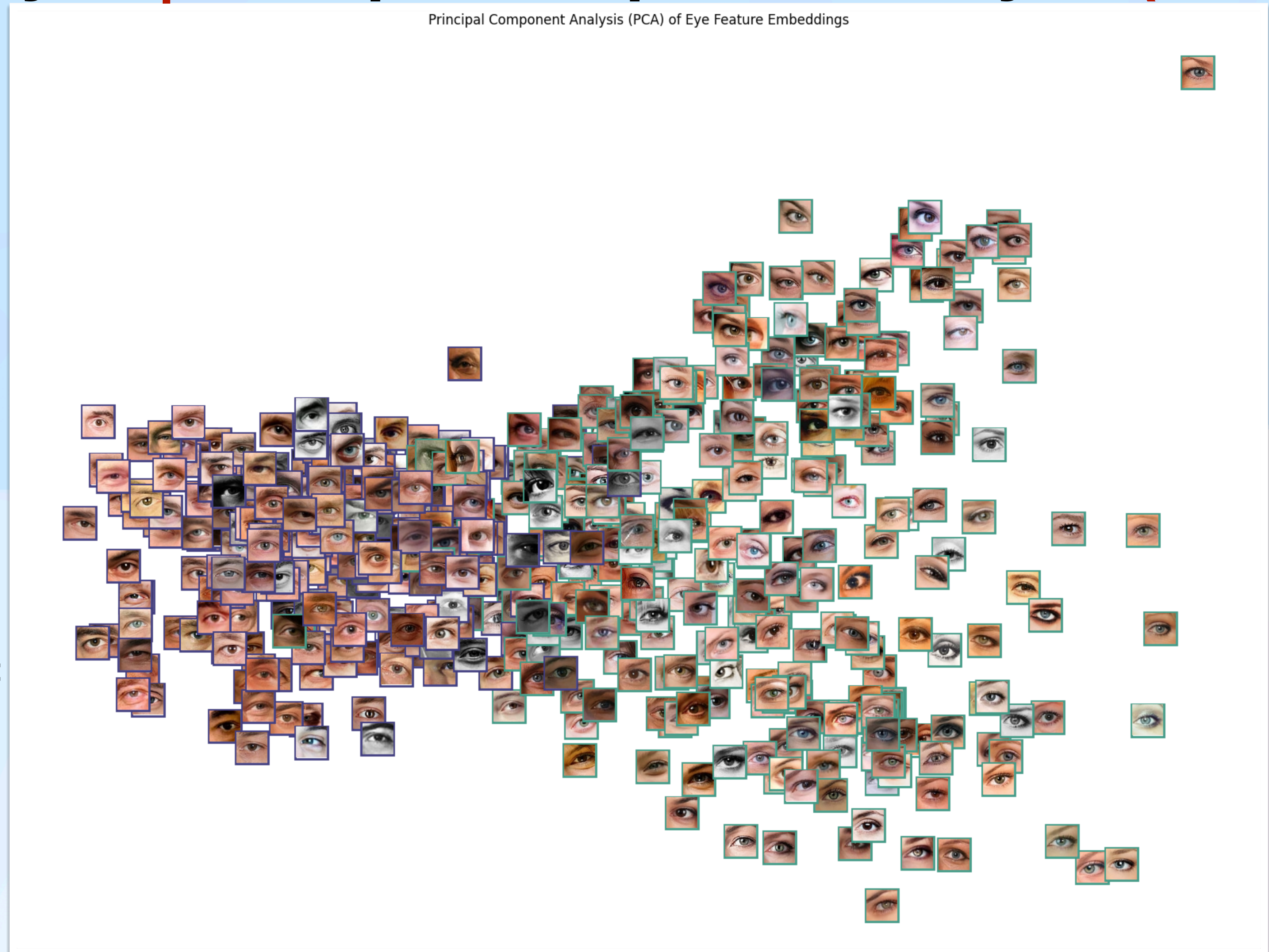
Dense(1024)

Dense(1024) ←

Dense(2)

Explained Variance Ratios:

- PC1: 54.4%
- PC2: 3.5%



Model Analysis | Principal Component Analysis (PCA)

Model: **CNN**

Layer 16 - Last layer
before prediction.

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Conv2D(3,3)

Conv2D(3,3)

MaxPooling(2,2)

Flatten()

Dense(1024)

Dense(1024) ←

Dense(2)

Explained Variance Ratios:

- PC1: 54.4%
- PC2: 3.5%



Model Analysis | Principal Component Analysis (PCA)

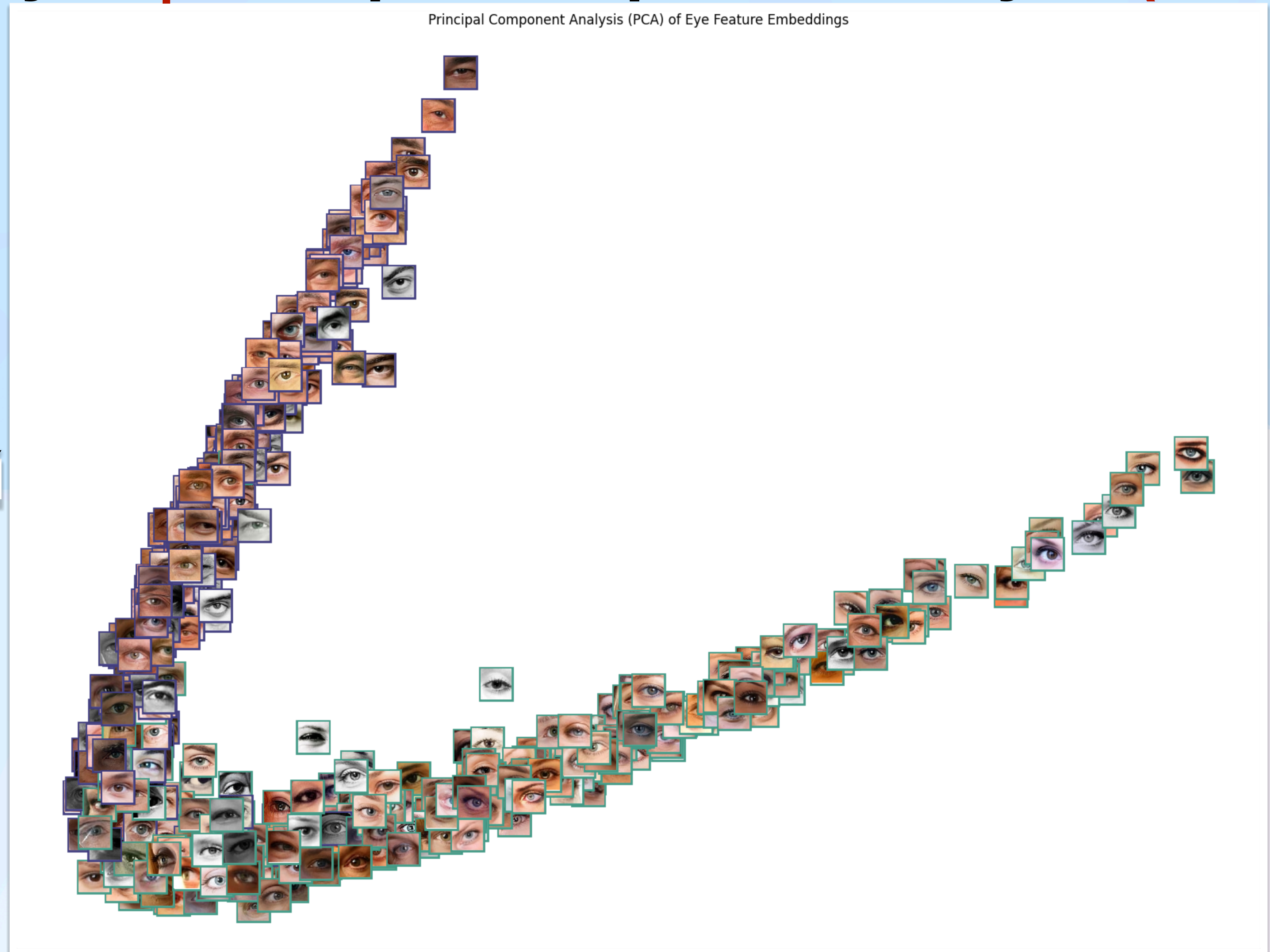
Model: DCNN

Layer 123 - Global
Average Pool before
prediction.



Explained Variance Ratios:

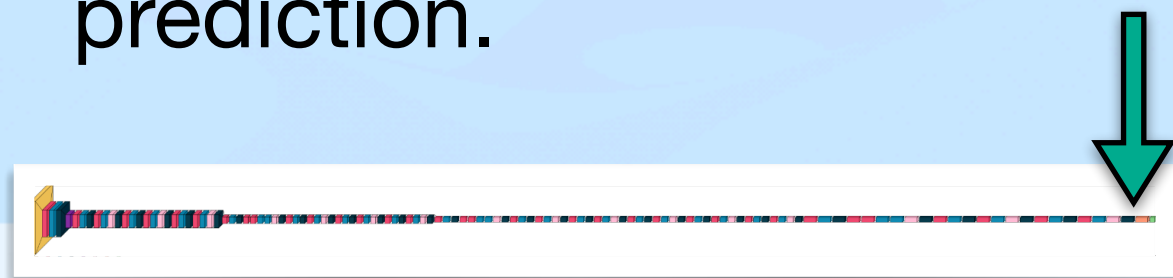
- PC1: 77.8%
- PC2: 10.6%



Model Analysis | Principal Component Analysis (PCA)

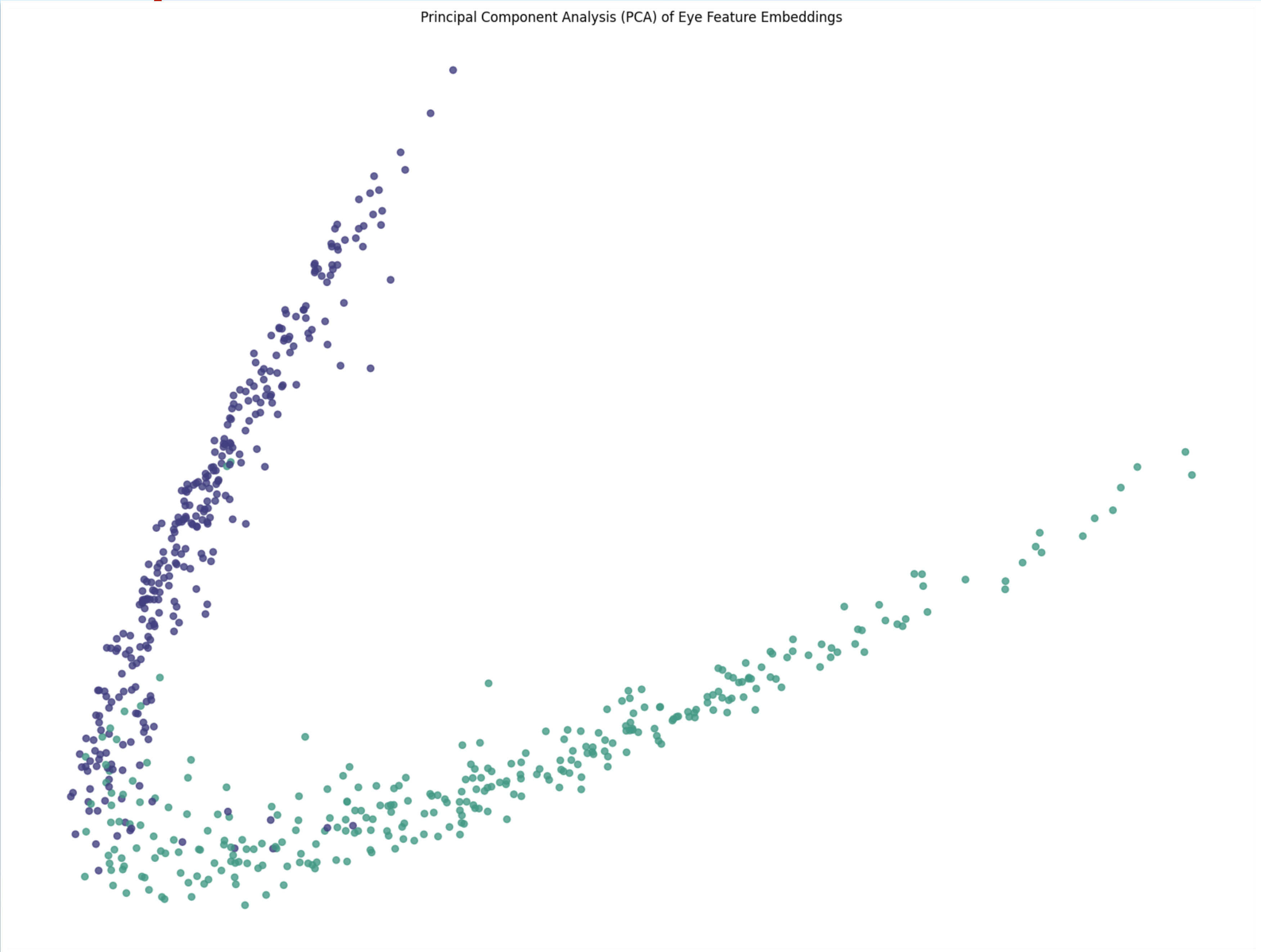
Model: DCNN

Layer 123 - Global Average Pool before prediction.

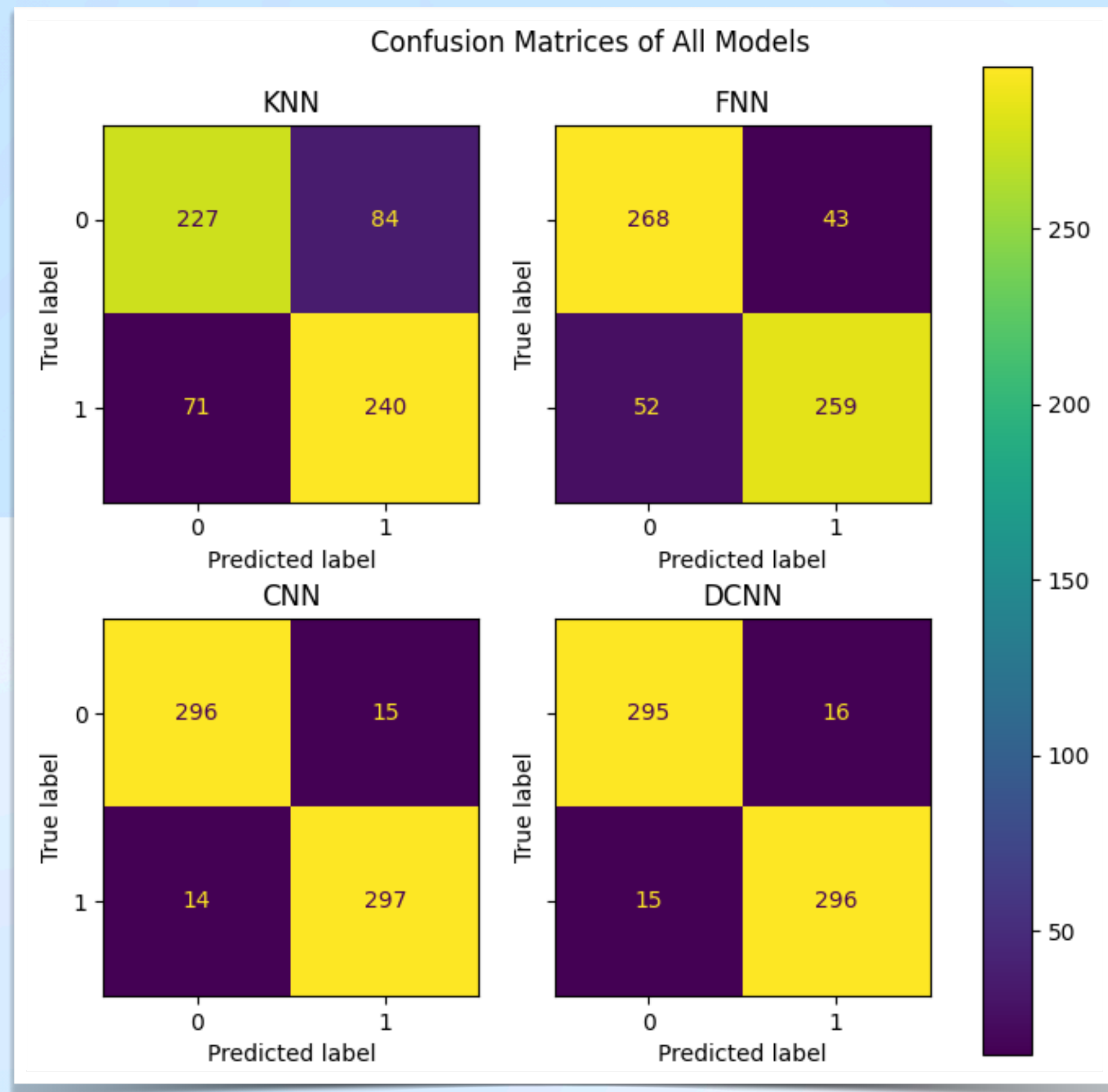


Explained Variance Ratios:

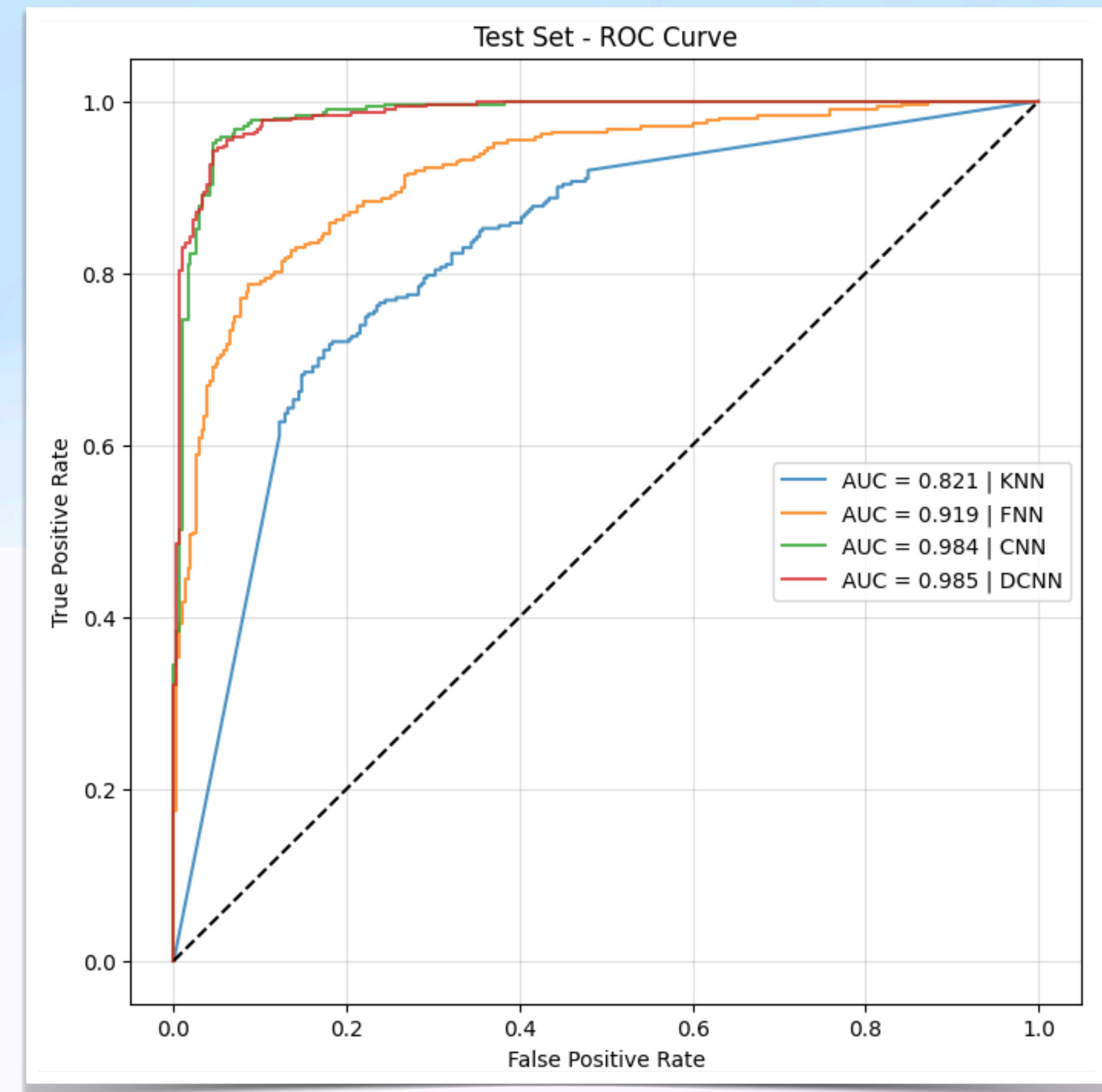
- PC1: 77.8%
- PC2: 10.6%



Results | Confusion Matrices & ROC Curve



- CNN and DCNN performed almost equally.
- FNN and even KNN performed surprisingly well.



Results | Evaluation Metric

- We again see CNN and DCNN almost equal.
- DCNN best on Val Set.
- CNN best overall on Test Set.

Non-Augmented Training:

- FNN improved performance.
 - Not a translation invariant model.
- CNN reduced performance.
- DCNN severely reduced performance.

	Augmented	Augmented.	Not Augmented	Not Augmented.
	Val Accuracy	Test Accuracy	Val Accuracy	Test Accuracy
Random Baseline	0.500000	0.500000	0.500000	0.500000
KNN	0.773848	0.750804	0.000000	0.000000
FNN	0.843516	0.847267	0.863880	0.861736
CNN	0.944266	0.953376	0.931404	0.914791
DCNN	0.945338	0.950161	0.908896	0.885852

In each column, the best metric is highlighted in green and second best is blue.

Misclassification Exploration (CNN)



Border indicates True Class: Teal = Female | Purple = Male

Common Characteristics:

- Hair
- Age (wrinkles)
- Weird angles
- Glasses
- Subtle eyelashes
- No eyebrows

Conclusion

Key Takeaways:

DCNN Efficacy: DCNNs extract essential features from the periocular region to differentiate between sexes, even with shallower networks.

Feature Extraction Insights:

Analysis of model filters and feature maps revealed:

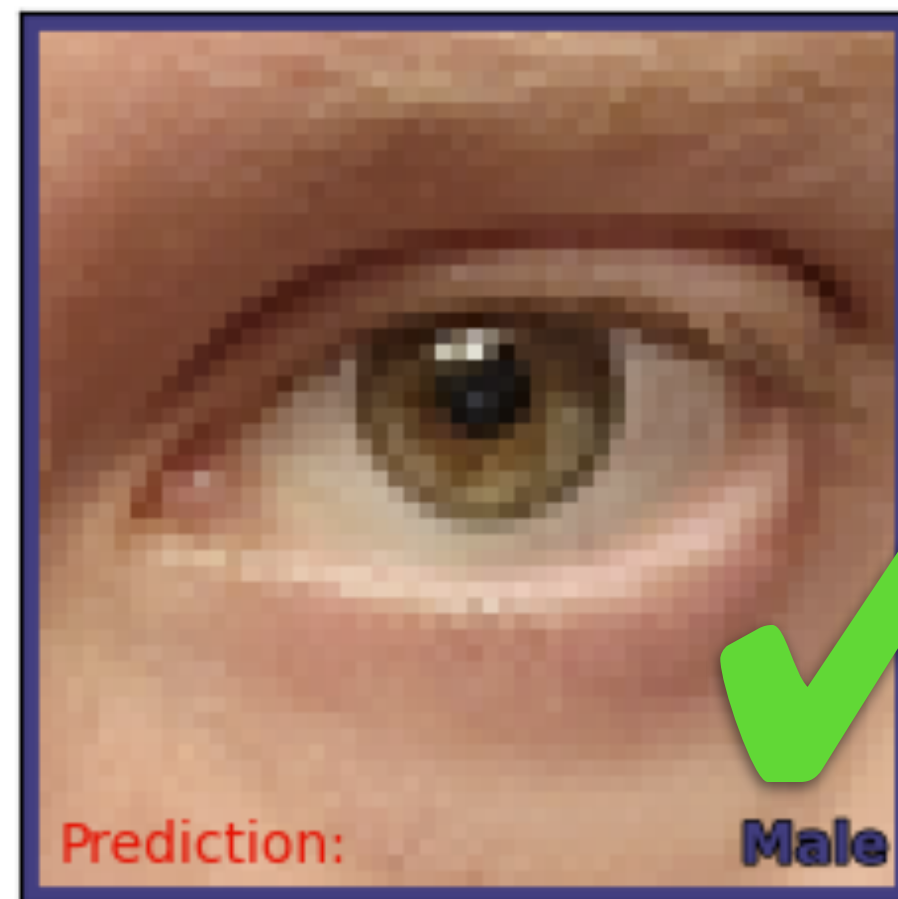
- Early convolutional layers focused on basic visual features such as edges and textures.
- Deeper layers captured more complex patterns and structures.
- Models focused on sexually distinct features for classification.

Data Augmentation's Impact:

- CNN-based models trained with image augmentation outperformed those without augmentation.
- Benefits include improved generalization and reduced overfitting.
- Necessary in deep learning with small datasets — increased size and diversity to the dataset improved performance.

Thank you!

Me



Wife

